FocusLens — Full Project Context Document

For use when resuming with a new AI session

WHAT IS THIS PROJECT?

FocusLens is a Chrome extension + Next.js web dashboard that tracks a student's focus level on any learning platform (YouTube, Coursera, Udemy, any LMS) in real time. When focus drops below a threshold (score < 60), an AI tutor widget pops up inside the page and offers concept re-explanations, quizzes, and personalized micro-lessons. The website dashboard shows heatmaps, daily reports, weekly reports, and topics studied — all powered by NVIDIA NIM AI API.

Built for: GDG Hacker Cup @ KNIT Sultanpur (PS 2 — "When Learning Becomes Exhaustion")

TECH STACK

Layer	Technology
Framework	Next.js 15 (App Router, Server Components)
Frontend	React 19, Tailwind CSS 3, motion/react (animations)
UI Theme	Neo-brutalism (bold borders, shadows, Space Mono)
Database	MongoDB via Mongoose
Auth	Custom JWT (httpOnly cookie `fl_token`)
AI	NVIDIA NIM API (OpenAI-compatible, llama-3.1-70b)
Extension	Chrome MV3 (content.js + background.js + overlay)
Fonts	Space Grotesk + Space Mono (Google Fonts)

ENVIRONMENT VARIABLES (.env.local)

```
MONGODB_URI=mongodb+srv://...          ← Paste your MongoDB Atlas URI here
NEXTAUTH_SECRET=random_32_char_string ← Any random string, min 32 chars
NEXTAUTH_URL=http://localhost:3000
GOOGLE_CLIENT_ID=...                   ← Optional: Google OAuth
GOOGLE_CLIENT_SECRET=...               ← Optional: Google OAuth
```

```
NVIDIA_API_KEY=nvapi-...              ← Get from build.nvidia.com
NVIDIA_BASE_URL=https://integrate.api.nvidia.com/v1
NVIDIA_MODEL=meta/llama-3.1-70b-instruct
```

PROJECT STRUCTURE

```
focuslens/
├── app/
|   ├── layout.tsx                  ✅ Root layout (fonts, metadata)
|   ├── globals.css                 ✅ Tailwind + neo-brutalism base styles
|   ├── page.tsx                    ✅ Landing page (hero, features,
testimonials, footer)
|   ├── login/page.tsx              ✅ Login page (email+password + Google)
|   ├── signup/page.tsx             ✅ Signup page
|   ├── dashboard/
|   |   ├── layout.tsx              ✅ Dashboard layout (sidebar nav, auth guard)
|   |   ├── page.tsx                ✅ Overview (stats, hourly heatmap, 30-day
calendar, topics, timeline)
|   |   ├── daily/page.tsx          ✅ Daily report (AI narrative + timeline +
insight pills)
|   |   ├── weekly/page.tsx         ✅ Weekly report (AI narrative + day bars +
topic breakdown)
|   |   └── topics/page.tsx         ✅ Topics studied (today + 30-day list with
scores)
|   └── api/
|       ├── auth/signup/route.ts    ✅ POST signup → creates user, sets JWT
cookie
|       ├── auth/login/route.ts     ✅ POST login → validates, sets JWT cookie
|       ├── auth/logout/route.ts    ✅ POST logout → clears cookie
|       ├── auth/me/route.ts        ✅ GET current user (used by extension)
|       ├── session/route.ts        ✅ POST signal from extension, PATCH end
session
|       └── chat/route.ts           ✅ POST AI chat (streaming SSE, used by
overlay widget)
├── components/
|   └── dashboard/
|       └── LogoutBtn.tsx           ✅ Contains: LogoutBtn, StatCard,
HeatmapClient, TopicBars, TodayTimeline
├── lib/
|   ├── db.ts                       ✅ MongoDB connection (singleton cached)
|   ├── ai.ts                       ✅ NVIDIA NIM client (OpenAI-compatible)
|   └── auth.ts                     ✅ JWT sign/verify/getSession from cookie
├── models/
|   ├── User.ts                     ✅ Mongoose schema: name, email, password,
```

```
provider
|   └── Session.ts                  ✅ Mongoose schema: userId, date, signals[],
scores, stats
├── public/extension/
|   ├── manifest.json               ✅ Chrome MV3 manifest
|   ├── content.js                  ✅ Tracks idle/scroll/speed, sends signals
every 15s
|   ├── background.js               ✅ Service worker: badge, overlay trigger,
auth redirect
|   ├── popup.html                  ✅ Extension popup UI (toggle, score display,
nav buttons)
|   └── overlay.js                  ✅ Injected AI chat widget (fab button + chat
panel + streaming)
├── package.json                    ✅
├── tailwind.config.js              ✅ Neo-brutalism colors, brutal shadows,
Space Mono
├── tsconfig.json                   ✅
├── next.config.mjs                 ✅
└── .env.local                      ✅ Template (fill in your values)
```

COMPLETED FEATURES ✅

Website / Dashboard

- ☑ Neo-brutalism landing page (navbar, hero, how-it-works, features, testimonials, CTA, footer)
- ☑ Login page (email+password, Google OAuth button)
- ☑ Signup page
- ☑ JWT auth (httpOnly cookie, 7 day expiry)
- ☑ Dashboard layout with sidebar (protected route, redirects to /login if no token)
- ☑ Overview page: 4 stat cards + hourly heatmap + 30-day calendar heatmap + topic bars + timeline
- ☑ Daily report: AI narrative (NVIDIA) + insight pills + session timeline
- ☑ Weekly report: AI narrative + day-by-day bars + topic breakdown + stat grid
- ☑ Topics page: today's topics + 30-day topic list with avg scores
- ☑ Logout button

API Routes

- ☑ POST /api/auth/signup
- ☑ POST /api/auth/login
- ☑ POST /api/auth/logout
- ☑ GET /api/auth/me
- ☑ POST /api/session (receive signals from extension)

- ☑ PATCH /api/session (end session)
- ☑ POST /api/chat (streaming AI tutor, SSE)

Chrome Extension

- ☑ Manifest V3
- ☑ Content script: idle timer, re-scroll detection, reading speed, focus score formula
- ☑ Background service worker: badge color/text, overlay injection, auth redirect
- ☑ Popup UI: toggle tracking on/off, live score display, nav buttons
- ☑ Overlay widget: floating fab, AI chat panel, streaming responses, auto-open on low score
- ☑ Auth check: if not logged in → opens login page

Database

- ☑ User model (email, password bcrypt hashed, name, provider)
- ☑ Session model (userId, date, signals[], avgScore, peakScore, lowestScore, alerts, quizzes)

WHAT IS LEFT / NOT DONE ❌

High priority (needed to demo)

- ☐ **Google OAuth actual implementation** — Currently only a button exists. Need to add:
  - `app/api/auth/google/route.ts` → redirect to Google
  - `app/api/auth/google/callback/route.ts` → exchange code, create user, set cookie
  - Use `passport-google-oauth20` or manually call Google OAuth2 API
- ☐ **Extension icon files** — Need `icon32.png` and `icon128.png` in `public/extension/`
  - Just create a simple yellow square with "FL" text or use any 32x32 / 128x128 PNG
- ☐ **Extension: score stored in chrome.storage** — In `content.js`, after getting score back from API, add:

```js
chrome.storage.local.set({ fl_score: focusScore })
```

This makes popup.html update in real time.

- ☐ **Dashboard import fixes** — The dashboard/page.tsx imports components like this:

```ts
import HeatmapClient from '@/components/dashboard/HeatmapClient'
import StatCard from '@/components/dashboard/StatCard'
```

But these are re-exports from LogoutBtn.tsx. Change all imports to:

```ts
import { HeatmapClient, StatCard, TopicBars, TodayTimeline } from '@/components/da
import LogoutBtn from '@/components/dashboard/LogoutBtn'
```

Medium priority (polish)

- [ ] **Quiz parsing in overlay** — When AI returns JSON quiz format, parse it and render clickable MCQ buttons instead of raw JSON text. The API already prompts for JSON when user asks to be quizzed. In overlay.js, after getting fullText, check if it starts with `{` and parse accordingly.
- [ ] **Mobile sidebar** — The bottom nav exists but active state highlighting is missing. Add `usePathname()` to highlight current route.
- [ ] **Session deduplication** — Currently a new session is created if `sessionId` is null. If the user refreshes mid-session, they'll get a duplicate. Fix by storing `fl_sessionId` in `chrome.storage.local` and reusing it.
- [ ] **Error boundaries** — Dashboard pages make DB calls that can throw. Wrap in try/catch with user-friendly empty states (partially done in daily/weekly).
- [ ] **Loading states** — Dashboard pages are Server Components so no spinner needed, but add `loading.tsx` files in each route folder for Suspense streaming.

Low priority (nice to have)

- [ ] **Real heatmap library** — Currently using a custom div grid. Can swap with `react-calendar-heatmap` for a proper GitHub-style calendar.
- [ ] **Educator view** — A separate route `/educator` showing all students' scores for a class (multi-user). Not built.
- [ ] **PWA / offline support** — Not implemented.
- [ ] **Rate limiting** — No rate limits on API routes. Add `upstash/ratelimit` if deploying publicly.
- [ ] **Email verification** — Signup doesn't verify email. Fine for hackathon.
- [ ] **Dark mode** — Not implemented (neo-brutalism is inherently light-mode).

HOW TO RUN

```bash
```

```
# 1. Install dependencies
cd focuslens
npm install

# 2. Fill in .env.local
# Set MONGODB_URI and NVIDIA_API_KEY (minimum required)

# 3. Run development server
npm run dev
# → http://localhost:3000

# 4. Load Chrome extension
# Open chrome://extensions
# Enable "Developer mode"
# Click "Load unpacked"
# Select the focuslens/public/extension folder
```

HOW THE FOCUS SCORE WORKS

```
idlePenalty   = min(idleSeconds / 120, 1) × 40   → max -40 pts for 2+ min idle
scrollPenalty = min(reScrollCount / 10, 1) × 30  → max -30 pts for 10+ re-scrolls
speedBonus    = min(readingSpeed / 200, 1) × 30   → max +30 pts for 200+ wpm

focusScore = clamp(100 - idlePenalty - scrollPenalty + speedBonus, 0, 100)
```

Score < 60 → AI widget auto-opens
Score < 40 → Red alert shown
Correct quiz answer → +8 pts (server-side, update session avgScore)

AI PROMPTS USED

Daily report prompt (lib context: one day of session data)

Asks NVIDIA llama-3.1-70b for a 3-paragraph personal coach report covering: what went well, what went wrong, and one concrete suggestion for tomorrow.

Weekly report prompt (lib context: 7 days of session data)

Asks for 4 paragraphs: week summary, patterns, problems, and next-week action plan.

Chat / overlay prompt (real-time, streaming)

System prompt instructs the model to: explain simply, quiz in JSON format, summarize in

bullets, use Socratic method when stuck. Keeps responses under 150 words for the small widget UI.

KEY DECISIONS MADE

1. **Custom JWT over NextAuth** — Simpler, no adapter needed, works with extension cookies easily.
2. **NVIDIA NIM API** — OpenAI-compatible, so `openai` npm package works with just `baseURL` changed.
3. **Server Components for dashboard** — DB calls happen on server, no client-side data fetching needed.
4. **SSE streaming for chat** — Overlay widget gets streamed tokens for a real-time feel.
5. **Chrome MV3** — Required for new extensions. Uses service workers not background pages.
6. **Neo-brutalism** — Chosen for visual distinctiveness in hackathon demo. Bold borders and yellow make it unmissable.

DEMO SCRIPT (for hackathon)

1. Open laptop, show landing page at localhost:3000
2. Click "Sign up free" → create account → redirect to dashboard (empty state)
3. Open Chrome extensions → show FocusLens is loaded → click extension icon → show popup
4. Navigate to a YouTube tutorial → toggle "Track this site" ON
5. Sit idle for 30s (pre-demo: lower the threshold to 20s in content.js for demo speed)
6. Score drops → overlay widget auto-opens with "Focus score dropped!" banner
7. Click "Quiz me" → AI generates quiz question → answer it → score recovers
8. Type a question: "Explain recursion simply" → watch AI stream response
9. Switch tab to dashboard → show hourly heatmap with the dip visible
10. Click "Daily report" → show AI-generated narrative with the exact session described
11. Show weekly report (pre-populate some fake sessions in MongoDB for demo)

Total demo time: ~90 seconds

MONGODB SCHEMA SUMMARY

Users collection

```json
{ "_id": ObjectId, "name": "Aarav Singh", "email": "aarav@example.com",
  "password": "$2b$12$...", "provider": "credentials", "createdAt": Date }
```

Sessions collection

```json
{ "_id": ObjectId, "userId": ObjectId, "date": "2026-04-12",
  "startTime": Date, "endTime": Date, "platform": "youtube.com",
  "topic": "Dynamic Programming - MIT OCW",
  "signals": [{ "timestamp": Date, "idleSeconds": 45, "reScrolls": 3,
                "readingSpeed": 120, "focusScore": 58, "pageTitle": "...", "pageUrl"
  "avgScore": 67, "peakScore": 89, "lowestScore": 42,
  "alerts": 2, "quizzes": 3, "quizCorrect": 2, "totalMinutes": 47 }
```

Last updated: April 2026 | Built for GDG Hacker Cup @ KNIT Sultanpur